

Module 1:

1. Đầu vào (Inputs):

- Multi-view RGB Images: Các bức ảnh chụp cảnh tĩnh và vật thể mục tiêu từ nhiều vị trí, và góc độ khác nhau

2. Đầu ra (Outputs)

- Thông số máy ảnh (Camera Parameters):

- Ma trận thông số nội suy (Intrinsics K): Tiêu cự (f_x, f_y), điểm chính quang học (c_x, c_y), hệ số méo thấu kính.
- Ma trận thông số ngoại suy (Extrinsic $[R | t]$): Vị trí tâm máy ảnh o và ma trận xoay góc nhìn R của từng bức ảnh trong hệ tọa độ thế giới (World coordinate system).

- **Sparse Point Cloud:** Tập hợp tọa độ 3D (X, Y, Z) kèm màu sắc và độ bất định của các điểm đặc trưng bề mặt, phục vụ khởi tạo cho 3DGS.

- **Binary 2D Masks:** Tập hợp ảnh mặt nạ nhị phân kích thước $H \times W$ tương ứng với từng góc chụp, phân tách rõ vùng điểm ảnh thuộc foreground (vật thể tương tác) và background

- **Point-to-Image Visibility Graph:** Danh sách theo dõi điểm 3D nào được quan sát thấy bởi những bức ảnh nào (phục vụ bộ chọn góc nhìn camera sau này).

3. Cách module hoạt động

Nhánh 1: Tái tạo hình học qua Structure-from-Motion (SfM)

1. Trích xuất và so khớp đặc trưng (Feature Extraction & Matching): Sử dụng SIFT (hoặc SuperPoint) để tìm các điểm mốc cục bộ bất biến trên từng ảnh, sau đó dùng thuật toán Nearest Neighbor kết hợp kiểm tra tính nhất quán hình học epipolar (RANSAC) để liên kết các cặp điểm tương đồng giữa các ảnh.
2. Ước lượng tham số và mây điểm thưa (Incremental Reconstruction):
 - Khởi tạo với một cặp ảnh có khoảng cách nền chuẩn (baseline) phù hợp.
 - Dùng thuật toán PnP (Perspective-n-Point) và Triangulation để tính vị trí camera tiếp theo và vị trí các điểm mốc 3D.
 - Chạy tối ưu hóa sai số góc nhìn lại toàn cục thông qua Bundle Adjustment (BA) để chốt chính xác ma trận K , $[R | t]$, tọa độ mây điểm thưa, đồng thời lưu lại đồ thị quan sát **Visibility Graph**.

Nhánh 2: 2D Semantic/Instance Segmentation

1. Sử dụng mô hình 2D segmentation đã được pretrain (Yolov11 Seg) để tạo ra mask cho vật thể

Module 2:

1. Nhánh Scene-GS:

a. Đầu vào (Inputs):

1. Camera Poses (Intrinsics K , Extrinsics $[R | t]$) (Module 1)
2. (Sparse Point Cloud từ SfM (Module 1).

b. Đầu ra (Outputs):

- Checkpoint mô hình toàn cảnh **Scene-GS.ply** (chứa vị trí p , tỉ lệ s , góc xoay R , độ đục α , và hệ số màu cầu Spherical Harmonics của các hạt Gaussian toàn cảnh).
- Phân bố hình học thô của vật thể mục tiêu, làm gốc khởi tạo cho Module 3

c. Cách module hoạt động

Bước 1: Phân tách tập ảnh (View Splitting)

- Tập ảnh được chia làm 2: Ảnh toàn cảnh (Global Context) và Ảnh cận cảnh tập trung vào vật thể (Close-up views).
- Nhánh Scene-GS nạp: 100% ảnh toàn cảnh + 30% - 50% ảnh cận cảnh của vật thể (được lấy mẫu đều). Việc đưa một lượng ảnh vật thể vào giúp Scene-GS học được khung hình học bao quát của vật, tránh vùng này bị rỗng hoàn toàn.

Bước 2: Khởi tạo mô hình (Initialization)

- Tải toàn bộ mây điểm thưa SfM làm vị trí ban đầu (p_k) cho các hạt Gaussian.

Bước 3: Vòng lặp tối ưu hóa (Optimization Loop - 20,000 iters)

- Áp dụng thuật toán 3DGS tiêu chuẩn (hoặc cải tiến tích hợp hàm mất mát Normal/Distortion của GOF nếu muốn mesh nền sạch):
 - Chiếu Gaussian lên mặt phẳng ảnh, thực hiện Alpha blending tính màu.
 - Tính mất mát trắc quang (Photometric loss): $L_c = (1-\lambda)L_1 + \lambda L_{D-SSIM}$.
 - Tự động nhân đôi (Cloning) hoặc chia tách (Splitting) các hạt Gaussian có độ dốc vị trí lớn hơn ngưỡng τ
 - Cắt tỉa (Pruning) các hạt có độ đục α quá thấp.

Bước 4: Xuất kết quả

- Dừng ở bước 20k. Lưu file **Scene-GS.ply**

2. Nhánh ROI 3D

a. Đầu vào (Inputs)

- **Binary 2D Masks từ Module 1:** Tập hợp các ảnh mask nhị phân tương ứng với các ảnh, phân tách vùng vật thể thao tác và phông nền.
- **Thông số máy ảnh (Camera Parameters) từ Module 1:**
 - Ma trận nội suy (Intrinsics K): Tiêu cự (f_x, f_y), tâm quang học (c_x, c_y)

- Ma trận ngoại suy (Extrinsics $[R | t]$): Tọa độ tâm máy ảnh và hướng quay trong hệ tọa độ thế giới.
- **Sparse Point Cloud & Visibility Graph từ Module 1:** Tọa độ 3D của các điểm đặc trưng kèm danh sách ID góc nhìn quan sát thấy từng điểm.

b. Đầu ra (Outputs)

- **3D Bounding Box:**
 - Axis-Aligned Bounding Box - AABB: $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$
- **Sub 3D bounding Box bên trong**
 - Sub Axis-Aligned Bounding Box - AABB: $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ (có class 1)
 - Sub Axis-Aligned Bounding Box - AABB: $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ (có class 0)
- **Danh sách góc nhìn tối ưu cho ROI (ROI Camera Subset):** Danh sách các camera có tầm nhìn trực tiếp, đạt chuẩn độ phân giải và độ phủ lên vật thể để phục vụ cho train Object-GS ở Module 3

c. Cách hoạt động

Bước 1: 2D-to-3D Back-projection & Inlier Filtering

1. **Lọc điểm từ Mây điểm thừa (SfM Points Filtering):**
 - Một điểm 3D được giữ lại nếu hình chiếu của nó rơi vào bên trong mask 2D (Mask value = 1) trên phần lớn các góc nhìn quan sát (dùng ngưỡng đồng thuận đa góc nhìn, ví dụ ≥ 3 góc nhìn để loại bỏ điểm nhiễu lơ lửng bên ngoài)

Bước 2: 3D Bounding Box Generation

1. **Khớp hộp bao (Box Fitting):**
 - Tính toán giá trị cực tiểu và cực đại theo từng trục tọa độ (X, Y, Z) từ tập điểm 3D đã lọc để sinh ra AABB.
2. **Mở rộng vùng biên (Margin Inflation):**
 - 3D bounding box được cộng thêm một hệ số đệm an toàn ϵ (ví dụ: mở rộng 5% - 10% kích thước mỗi chiều)
 - Vùng đệm này đóng vai trò quan trọng: ngăn chặn việc cắt đứt đột ngột các hạt Gaussian có tâm nằm sát biên nhưng bán kính tỏa ra ngoài, giúp Module 3 huấn luyện vùng tiếp giáp không bị nứt vỡ hình học.

Bước 3: Voxel Partitioning & View Assignment

1. **Thiết lập 3D Voxel Grid:**
 - Chia AABB thành lưới voxel có kích thước cạnh v_size
 - Phân loại từng ô lưới voxel đó (ô lưới đó có class là 1 nếu số lượng điểm 3D nằm trong ô lưới đó $>$ một ngưỡng nhất định và class 0 là ngược lại)

- Sử dụng thuật toán gom cụm để gom các ô voxel đó tạo thành AABB_sub dựa trên Geometric Variance hoặc độ cong cao
- 2. **Xác định tập Camera cho Object ROI (ROI-Focused Camera Selection):**
 - Kiểm tra hình chiếu của AABB 3D và Sub AABB 3D lên mặt phẳng ảnh của từng camera.
 - Chọn lọc các camera thỏa mãn:
 - AABB hoặc Sub AABB nằm trọn vẹn trong khung hình quan sát (FoV)
 - Tỷ lệ diện tích chiếu của vật thể trên tổng diện tích ảnh lớn hơn ngưỡng quy định (đảm bảo camera chụp đủ gần, không lấy góc nhìn xa tít gây mờ).
 - Danh sách ID camera này được đóng gói để cấp quyền nạp dữ liệu riêng cho Module 3

Module 3:

1. Đầu vào (Inputs)

- **Scene-GS.ply** từ Module 2: Chứa toàn bộ các hạt 3D Gaussian của toàn bộ không gian đã được tối ưu sơ bộ sau 20k bước.
- **3D Bounding Box (3D AABB/OBB) và nhãn Voxel** từ Module 3: Xác định chính xác phạm vi hình học $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ bao quanh vật thể.
- **Tập ảnh cận cảnh và thông số góc nhìn (ROI Camera Subset)** từ Module 2: Các góc máy chụp gần, có độ phân giải cao và tỷ lệ che phủ lớn đối với vật thể.
- **Tập ảnh toàn cảnh (Coarse Views)** từ Module 1: Dùng để duy trì thông tin nền, tránh hiện tượng trôi dạt tại đường biên.

2. Đầu ra (Outputs)

- **Object-GS.ply**: Mô hình Gaussian chuyên biệt cho vật thể với mật độ hạt cao, cấu trúc sắc nét tại các chi tiết hình học nhỏ (quai cầm, lỗ rỗng, gờ cạnh).
- **Mô hình 3DGS Hoàn chỉnh (Composed-3DGS.ply)**: Mô hình kết hợp hợp nhất toàn bộ không gian:
 - Bên trong vùng ROI: Được lấp đầy bởi các hạt Gaussian vi mô siêu nét từ **Object-GS**.
 - Bên ngoài vùng ROI: Giữ nguyên các hạt Gaussian vĩ mô từ **Scene-GS** để tối ưu tải tính toán.

3. Cách thức hoạt động chi tiết (Pipeline Execution)

Giai đoạn 1: Khởi tạo Object-GS từ Scene-GS (Contextual Initialization)

- **Object-GS** được sao chép nguyên trạng từ toàn bộ mô hình **Scene-GS**. Bối cảnh nền có sẵn giúp quá trình tối ưu hóa vùng biên của vật thể ổn định và bám sát thực tế.

Giai đoạn 2: Tối ưu hóa tập trung vào ROI (Targeted ROI Training - 30,000 iters)

Huấn luyện nhánh **Object-GS** bằng tập ảnh chọn lọc góc cận với các ràng buộc không gian đặc biệt:

- **Tối ưu trên toàn ảnh (Full-image Rendering Loss)**: Quá trình tính hàm mất mát vẫn diễn ra trên toàn bộ khung hình để cập nhật cả các hạt nền xung quanh, tránh tạo ra viền cắt đột ngột.
- **Khu biệt hóa tăng sinh hạt (Confined Densification - 15,000 iters đầu)**:
 - Tiêu chí nhân đôi (Clone) hoặc chia tách (Split) hạt Gaussian chỉ được kích hoạt nếu tâm của hạt đó **nằm bên trong phạm vi 3D AABB của vật thể**.
 - Các hạt nằm ngoài hộp bao sẽ bị đóng băng cấu trúc (không sinh thêm hạt), tập trung toàn bộ dung lượng bộ nhớ VRAM và số lượng hạt vào các chi tiết vi mô của vật thể.
- **Cắt tỉa chủ động (Aggressive Pruning)**: Lọc bỏ các hạt thừa kế thừa từ **Scene-GS** nhưng không đóng góp vào việc tái tạo chi tiết vật thể hoặc có độ đục α quá nhỏ, giảm thiểu tối đa kích thước mô hình.

Giai đoạn 3: Ghép nối không gian (Scene-Objects Composition)

1. **Loại bỏ hạt thô cũ**: Quét toàn bộ mô hình **Scene-GS** ban đầu; mọi hạt Gaussian có tọa độ tâm p_k rơi vào bên trong hộp bao 3D AABB của vật thể đều bị xóa bỏ.

2. **Trích xuất hạt mịn mới:** Quét mô hình **Object-GS** vừa huấn luyện xong; chỉ giữ lại những hạt Gaussian có tâm p_k nằm bên trong hộp bao 3D AABB.
3. **Hợp nhất danh sách hạt (Concatenation):** Ghép nối mảng hạt mịn của **Object-GS** vào mảng hạt nền của **Scene-GS**.
4. **Kết quả:** Sinh ra **Composed-3DGS.ply** mang cấu trúc đa độ phân giải thích ứng (Adaptive Level-of-Detail): vùng vật thể đạt độ nét cực cao mà tổng số lượng hạt của toàn cảnh không bị quá tải. Mô hình này chính là đầu vào trực tiếp cho thuật toán dựng trường độ đục và trích xuất bề mặt lưới bằng **GOF**.

Module 4:

Trích xuất Mesh:

Áp dụng nguyên vẹn cơ chế trích xuất mesh của GOF (Gaussian Opacity Fields), không thay đổi thành phần nào.

1. Đầu vào (Inputs)

- **Mô hình 3DGS hoàn chỉnh (Merged-GS.ply)**: tập hạt Gaussian đã hợp nhất, mỗi hạt lưu vị trí μ , tỉ lệ s , góc xoay R , độ đục α và hệ số màu.
- **Camera Poses** của toàn bộ tập ảnh huấn luyện: cần thiết vì trường độ đục được đánh giá theo từng góc nhìn đã quan sát.

2. Đầu ra (Outputs)

- **Lưới tam giác (mesh.ply)**: tập đỉnh và mặt tam giác biểu diễn bề mặt liên tục của cảnh.
- **Tập chỉ số (metrics.json)**: số đỉnh, số mặt, trạng thái watertight, số thành phần liên thông, mức level set đã sử dụng.

3. Cách hoạt động

Bước 1: Sinh tập điểm mốc từ hạt Gaussian (Pivot Generation)

Với mỗi Gaussian, sinh ra chín điểm mốc: một điểm tại tâm và tám điểm tại các góc của hộp bao theo trục chính:

Hệ số $k=3$ tương ứng với việc bao phủ ba độ lệch chuẩn của phân bố Gaussian, đảm bảo hộp bao chứa gần như toàn bộ khối lượng xác suất của hạt.

Bước 2: Phân hoạch tứ diện Delaunay (Delaunay Tetrahedralization)

Toàn bộ tập điểm mốc được đưa vào thư viện CGAL để phân hoạch không gian thành các khối tứ diện, sao cho mỗi đỉnh lưới chính là một điểm mốc và các khối đạt độ cân đối tối ưu, tránh sinh ra khối dẹt méo gây sai số nội suy.

Bước 3: Đánh giá trường độ đục (Opacity Field Evaluation)

Thuật toán Marching Tetrahedra yêu cầu mỗi đỉnh lưới mang một giá trị vô hướng có dấu. GOF không huấn luyện thêm một trường riêng, mà sử dụng trực tiếp **độ đục** vốn đã có sẵn trong mô hình 3DGS.

Với mỗi điểm 3D, độ đục được tính theo từng góc nhìn thông qua phép giao tương minh giữa tia chiếu và các Gaussian, sau đó lấy giá trị nhỏ nhất trên toàn bộ các góc đã quan sát điểm đó:

$$O(x) = \min(r, t) O(o, r, t)$$

Giá trị vô hướng có dấu được định nghĩa bằng cách trừ đi ngưỡng phân định:

$$f(x) = O(x) - 0.5$$

Nghĩa là bề mặt vật thể được xác định tại tập hợp các điểm có độ đục bằng 0.5 điểm có độ đục lớn hơn được coi là nằm bên trong, nhỏ hơn là nằm bên ngoài.

Ý nghĩa của phép lấy cực tiểu. Đây chính là cơ chế khử nhiễu sẵn có của GOF: một điểm chỉ được coi là nằm trong vật thể nếu **mọi góc nhìn đều đồng thuận**. Chỉ cần một góc nhìn xuyên thấu qua điểm đó là nó bị đánh dấu rỗng. Nhờ vậy các hạt lơ lửng giữa không trung — vốn chỉ che khuất ở một vài góc nhất định — sẽ tự động bị loại bỏ mà không cần bước hậu xử lý riêng.

Ưu điểm của phương án này là không cần thêm tham số, không cần huấn luyện bổ sung, và tận dụng trực tiếp kết quả của mô hình 3DGS đã có. Đây là lý do nó phù hợp làm baseline.

Bước 4: Marching Tetrahedra kết hợp tìm kiếm nhị phân

Thuật toán duyệt qua từng **cạnh** của mỗi khối tứ diện và so sánh dấu của giá trị vô hướng tại hai đầu mút:

- Nếu f tại hai đầu **cùng dấu**, bề mặt không cắt qua cạnh đó, bỏ qua.
- Nếu **trái dấu**, bề mặt cắt qua một điểm nằm giữa hai đầu mút, cần xác định vị trí chính xác của điểm cắt.

Các điểm cắt trong cùng một khối tứ diện được nối lại thành tam giác, và tập hợp toàn bộ tam giác tạo thành lưới bề mặt.

Cải tiến của GOF ở khâu tìm điểm cắt. Marching Tetrahedra tiêu chuẩn giả định giá trị biến thiên tuyến tính dọc theo cạnh, do đó chỉ cần một phép nội suy tuyến tính là ra vị trí điểm cắt. Giả định này không đúng với trường độ đục, vốn có tính phi tuyến mạnh do bản chất tích lũy alpha-blending.

GOF thay bằng **tìm kiếm nhị phân trong từng khối tứ diện**. Về bản chất, tìm kiếm nhị phân mô phỏng hiệu quả việc lấy mẫu độ đục dày đặc dọc theo cạnh để định vị chính xác mặt đẳng trị. Chỉ sau vài vòng lặp, độ chính xác của bề mặt đã cải thiện đáng kể trong khi chi phí tính toán gần như không thay đổi, do đó bước này luôn được bật.

Bước 5: Trích xuất đa mức (Multi-level Extraction)

Do trường độ đục là liên tục, có thể trích xuất nhiều phiên bản lưới ở các mức level set khác nhau mà không cần chạy lại toàn bộ quy trình.

Với mục đích mô phỏng vật lý, mức 0.4 có thể được ưu tiên thay vì 0.5: lưới thu được nở ra một lượng nhỏ so với bề mặt thật, giúp giảm hiện tượng xuyên vật (interpenetration) trong quá trình tính toán va chạm của động cơ vật lý.

Module 5:

Đầu vào (Inputs)

- **Raw Mesh từ Module 4:** Lưới tam giác được trích xuất từ mô hình GS thông qua Gaussian Opacity Field và Marching Tetrahedra. Mesh gồm tập vertex và triangle biểu diễn bề mặt liên tục của vật thể.
- **3D ROI Module 2:** Vùng không gian xác định vật thể mục tiêu và các khu vực có khả năng tương tác hoặc chứa các chi tiết hình học quan trọng. Các thông tin ROI này được sử dụng để quyết định mức độ bảo toàn và đơn giản hóa mesh.

2. Đầu ra (Outputs)

- **Processed Mesh:** Mesh sau khi đã loại bỏ các thành phần nhiễu, sửa lỗi topology và kiểm tra tính hợp lệ.
- **Adaptive Meshes:** Các mesh có mật độ hình học cao tại các vùng Sub-ROI và giảm dần mật độ polygon tại các vùng ít quan trọng.
- **Collision Mesh:** Phiên bản mesh được tối ưu để đưa vào Physics Engine phục vụ collision detection.
- **Mesh Metrics:** Lưu các thông số như số vertex, số triangle, số connected components, số boundary edges, trạng thái watertight và tỷ lệ giảm polygon.

3. Cách module hoạt động

Bước 1: Làm sạch dữ liệu Mesh (Mesh Cleaning)

Raw Mesh từ Module 4 được đưa làm sạch nhằm loại bỏ các lỗi hình học có thể xuất hiện trong quá trình trích xuất bề mặt. Sử dụng **Vertex Deduplication** để phát hiện các vertex có tọa độ trùng hoặc gần trùng nhau trong một ngưỡng **Epsilon** và hợp nhất chúng thành một vertex duy nhất. Sau đó sử dụng **Degenerate Triangle Removal** để loại bỏ những tam giác có diện tích bằng hoặc gần bằng 0, có hai hoặc nhiều đỉnh trùng nhau hoặc có cấu trúc hình học không hợp lệ. Việc này giúp giảm các phần tử không cần thiết trước khi thực hiện các bước xử lý tiếp theo.

Bước 2: Loại bỏ các thành phần Mesh nhiễu

Sau khi loại bỏ vertex và triangle không hợp lệ, mesh được biểu diễn dưới dạng một đồ thị topology trong đó các triangle được liên kết với nhau thông qua các cạnh chung. Sử dụng **Connected Component Analysis** để tìm các thành phần liên thông trong mesh. Thành phần chính thường tương

ứng với bề mặt vật thể, trong khi những component rất nhỏ và nằm tách biệt có thể là các mảnh nhiễu phát sinh trong quá trình reconstruction. Những component có số lượng triangle hoặc diện tích nhỏ hơn một ngưỡng threshold sẽ được loại bỏ. Tuy nhiên, các component có ngưỡng thấp nằm trong vùng ROI không được xóa chỉ dựa trên kích thước nếu chúng có khả năng đại diện cho các chi tiết hình học quan trọng của vật thể.

Bước 3: Phát hiện và sửa lỗi Topology

Mesh sau khi lọc tiếp tục được kiểm tra topology thông qua **Boundary Edge Detection**. Một edge chỉ xuất hiện trong một triangle được xác định là boundary edge và tập hợp các boundary edge liên tiếp được gom thành boundary loop. Các loop nhỏ được xử lý bằng **Boundary Loop Triangulation** để tạo các triangle mới lấp kín lỗ thủng. Sau khi sửa, mesh được kiểm tra lại về manifoldness, orientation của normal và số lượng boundary edges. Mục tiêu của bước này là tạo ra một mesh có topology ổn định và ưu tiên đạt trạng thái **watertight**, vì mesh bị hở có thể gây sai lệch khi được sử dụng làm geometry cho collision.

Bước 4: Xác định vùng ROI trên Mesh

Sau khi mesh đã đạt trạng thái topology hợp lệ, **3D ROI** được xác định từ Module 2 và ánh xạ vào không gian của mesh. ROI được sử dụng làm vùng tham chiếu để xác định mức độ chi tiết cần thiết của mesh. Từ ROI ban đầu, không gian ROI được chia thành các **Sub-ROI** tương ứng với các mức độ chi tiết khác nhau của mesh. Với mỗi triangle, tính centroid:

$$C_i = (v_1 + v_2 + v_3) / 3$$

Centroid được sử dụng để xác định triangle thuộc ROI hoặc Sub-ROI nào. Các Sub-ROI này không trực tiếp quyết định việc xóa triangle mà đóng vai trò **tham chiếu không gian** để lựa chọn mức độ phân giải phù hợp ở các bước tiếp theo.

Bước 5: Tạo các Adaptive Meshes bằng QEM

Từ **Raw Mesh** sau bước topology repair, thuật toán **Quadric Error Metrics (QEM)** kết hợp với **Edge Collapse** được sử dụng để tạo ra nhiều phiên bản mesh có mức độ phân giải khác nhau. QEM xây dựng một ma trận quadric cho mỗi vertex nhằm biểu diễn sai số hình học khi vertex đó được thay đổi vị trí. Với mỗi edge, thuật toán tính chi phí khi gộp hai vertex thành một vertex mới. Edge có chi phí thấp hơn được ưu tiên collapse vì việc gộp edge đó gây ra ít sai lệch hình học hơn.

Quá trình simplification được thực hiện với nhiều ngưỡng τ_i . Mỗi ngưỡng tương ứng với một mức độ simplification và tạo ra một mesh mới:

$$M_0 \xrightarrow{\tau_1} M_1 \xrightarrow{\tau_2} M_2 \xrightarrow{\tau_3} \dots \xrightarrow{\tau_n} M_n$$

Trong đó M_0 là Raw Mesh, còn M_i là mesh sau khi thực hiện một mức độ edge collapse tương ứng với τ_i . Khi mức độ simplification tăng, số lượng triangle giảm và mesh trở nên thô hơn.

QEM computation → **Edge cost calculation** → **Minimum-cost edge selection** → **Edge Collapse** → **Topology update**

Bước 6: Liên kết các Mesh với các Sub-ROI

Mỗi mức mesh được liên kết với một phạm vi Sub-ROI tương ứng. Mesh có độ phân giải cao được sử dụng cho các Sub-ROI yêu cầu bảo toàn nhiều chi tiết, trong khi mesh có độ phân giải thấp hơn được sử dụng cho các Sub-ROI có thể chấp nhận mức simplification lớn hơn.

Có thể biểu diễn tập mesh và Sub-ROI tương ứng là:

Có thể biểu diễn tập mesh và Sub-ROI tương ứng là:

$$\mathcal{M} = \{M_1, M_2, \dots, M_n\}$$

$$\mathcal{R} = \{R_1, R_2, \dots, R_n\}$$

với mỗi cặp:

$$(M_i, R_i)$$

biểu diễn một **mức độ phân giải mesh** và **vùng ROI tương ứng**. Như vậy, QEM tạo ra các mức mesh, trong khi Sub-ROI cung cấp thông tin không gian để hệ thống lựa chọn mức mesh phù hợp với ROI thực tế.

Bước 7: So khớp ROI hiện tại với Sub-ROI

Khi vùng ROI của tác vụ thay đổi, hệ thống sử dụng ROI mới để tìm mức mesh phù hợp trong tập Adaptive Meshes đã được tạo

Sau đó tính tỷ lệ overlap:

$$Overlap_i = \frac{Vol(ROI \cap R_i)}{Vol(ROI)}$$

Sub-ROI có giá trị $Overlap_i$ lớn nhất được xem là vùng phù hợp nhất với ROI hiện tại:

$$i^* = \arg \max_i Overlap_i$$

-> Mesh tương ứng được lựa chọn:

$$\mathbf{M}_{selected} = \mathbf{M}_{i^*}$$

Bước 8: Kiểm tra sai số hình học

Sau khi lựa chọn $\mathbf{M}_{selected}$, mesh được so sánh với Raw Mesh để đánh giá mức độ sai lệch hình học bằng **Hausdorff Distance** hoặc **Chamfer Distance**.

Sai số tại vùng ROI được đánh giá riêng:

$$E_{ROI} = D(M_{selected}^{ROI}, M_{raw}^{ROI})$$

Đặc biệt, sai số tại vùng ROI được kiểm soát chặt hơn so với các vùng ngoài ROI: $E_{ROI} < \tau_{ROI}$

trong khi Non-ROI có thể cho phép sai số lớn hơn: $\tau_{NonROI} > \tau_{ROI}$

Nếu sai số tại ROI vượt ngưỡng τ_{ROI} , hệ thống chuyển sang mesh có **độ phân giải cao hơn** trong tập Adaptive Meshes và thực hiện đánh giá lại. Quá trình này tiếp tục cho đến khi tìm được mesh đáp ứng yêu cầu sai số tại vùng ROI.

Bước 9: Xuất Collision Mesh

Sau khi hoàn thành cleaning, topology repair và adaptive simplification, mesh được xuất thành `processed_mesh.ply`. Từ mesh này tạo ra **Collision Mesh** dành riêng cho Physics Engine. Các metric của mesh trước và sau simplification được lưu vào `metrics.json`.

Module 6:

1. Đầu vào (Inputs)

- **Collision Mesh từ Module 5:** Mesh đã được làm sạch, sửa topology và tối ưu mật độ theo ROI.
- **Visual Mesh:** Mesh sử dụng để hiển thị vật thể trong môi trường mô phỏng.
- **Physical Parameters:** Các thông số như mass, inertia, friction, damping và contact parameters.
- **Task Configuration:** Quỹ đạo hoặc trạng thái mục tiêu để thực hiện nhiệm vụ tương tác với vật thể.

2. Đầu ra (Outputs)

- **Physics Simulation Environment:** Môi trường mô phỏng chứa tương tác vật thể.
- **Collision/Contact Information:** Vị trí contact, contact normal, penetration depth và lực tiếp xúc.
- **Robot/Object Trajectory:** Quỹ đạo chuyển động của sự tương tác với vật thể trong quá trình mô phỏng.
- **Physics Metrics:** Simulation time, collision accuracy, interpenetration, contact stability và computational cost.

3. Cách module hoạt động

Bước 1: Khởi tạo môi trường Physics Engine

Đầu tiên xây dựng môi trường mô phỏng bằng **MuJoCo**. Hệ thống thiết lập hệ tọa độ, gravity, timestep và các thông số solver. Mô hình robot và vật thể được đặt vào cùng hệ tọa độ với mesh được tái dựng từ các module trước nhằm đảm bảo vị trí và orientation của vật thể trong simulation tương ứng với kết quả reconstruction.

Bước 2: Nạp Visual Mesh và Collision Mesh

Visual Mesh được sử dụng cho rendering nhằm hiển thị hình dạng vật thể, trong khi **Collision Mesh từ Module 5** được sử dụng làm geometry cho collision detection. Việc tách hai loại mesh giúp không cần sử dụng toàn bộ mesh có độ phân giải cao cho physics.

Bước 3: Thiết lập thuộc tính vật lý

Các thuộc tính vật lý được gán cho robot và vật thể bao gồm khối lượng, moment of inertia, friction coefficient và damping. Các tham số contact được thiết lập để Physics Engine có thể tính toán phản lực khi robot tiếp xúc với vật thể. Những tham số này được giữ cố định khi so sánh các loại mesh khác nhau để đảm bảo kết quả đánh giá chỉ phản ánh sự khác biệt do mesh.

Bước 4: Thiết lập Object Interaction

Robot được điều khiển từ trạng thái ban đầu đến vùng tương tác dựa trên **task configuration**. Quỹ đạo được thiết lập để robot tiếp xúc với các vùng thuộc ROI. Cùng một trajectory được sử dụng cho các phiên bản mesh khác nhau nhằm đánh giá khả năng duy trì hình học phục vụ nhiệm vụ của Adaptive Meshes.

Bước 5: Broad-phase Collision Detection

Tại mỗi timestep, Physics Engine sử dụng **bounding box quanh geometry** để nhanh chóng loại bỏ các cặp geometry không có khả năng va chạm (2 hộp không chồng lên nhau -> không va chạm). Chỉ các cặp có bounding box giao nhau mới được chuyển sang **Narrow-phase Collision Detection**, giúp giảm số phép kiểm tra hình học cần thực hiện.

Bước 6: Narrow-phase Collision Detection

các cặp geometry có khả năng collision được kiểm tra chính xác để xác định có xảy ra collision hay không. Với geometry dạng convex, các thuật toán như **GJK (Gilbert–Johnson–Keerthi)** - một thuật toán dùng trong **collision detection** để kiểm tra xem **hai vật thể dạng convex có đang giao nhau hay không**, đồng thời có thể dùng để tính **khoảng cách giữa chúng** => có thể được sử dụng, trong khi triangle mesh được xử lý dựa trên cấu trúc collision geometry. Kết quả là các cặp geometry đang va chạm hoặc tiếp xúc.

Bước 7: Contact Detection

Khi collision xảy ra, Physics Engine xác định **contact point**, **contact normal** và **penetration/distance**. Các thông tin này được sử dụng cho bước **contact resolution** và tính toán tương tác giữa robot và vật thể. Các contact tại ROI được theo dõi để đánh giá khả năng bảo toàn hình học của Adaptive Meshes.

Bước 8: Contact Resolution và Dynamics

Sau khi xác định contact, Physics Engine giải bài toán dynamics để tính phản lực và cập nhật trạng thái của các vật thể. Contact constraint được đưa vào solver để hạn chế robot xuyên qua bề mặt vật thể. Ma sát được tính dựa trên friction coefficient và vận tốc tương đối tại contact.

Kết quả sau mỗi timestep được sử dụng để cập nhật: $q_{t+1}, \dot{q}_{t+1}$

trong đó q biểu diễn trạng thái vị trí và $\dot{q}$ (đạo hàm của q) biểu diễn vận tốc của hệ thống.

Bước 9: Phát hiện Error Collision và Interpenetration

Để đánh giá chất lượng của mesh, hệ thống ghi nhận những trường hợp robot bị collision với vị trí mà mesh thực tế không nên có bề mặt, gọi là **Error collision**. Đồng thời, mức độ robot xuyên vào vật thể được đo thông qua **interpenetration depth** (độ xuyên lún giữa hai vật thể).

Hai chỉ số này đặc biệt quan trọng khi mesh được simplify. Nếu Non-ROI được giảm polygon quá mức, hình dạng bề mặt có thể bị biến dạng và gây ra collision sai; nếu ROI được bảo toàn tốt, các contact quan trọng đối với robot vẫn được duy trì.

Bước 10: So sánh các phương pháp Mesh

Để đánh giá hiệu quả của phương pháp, cùng một simulation task được thực hiện trên ba cấu hình mesh gồm **M(High)** là mesh có độ phân giải cao đồng đều, **M(Coarse)** là mesh được đơn giản hóa đồng đều và **M(Adaptive)** là **Adaptive Meshes** của đề tài. Các cấu hình sử dụng cùng robot, trajectory, physical parameters và simulation timestep. Kết quả được so sánh dựa trên **số polygon, simulation time, collision accuracy và interpenetration depth**.

Bước 11: Đánh giá hiệu quả

Hiệu quả của Adaptive Meshes được đánh giá dựa trên khả năng giảm chi phí tính toán trong khi vẫn bảo toàn hình học tại ROI.

$$\text{Reduction} = 1 - N(\text{adaptive})/N(\text{High})$$

- N_{Adaptive} = số polygon của Adaptive Meshes
- N_{HighRes} = số polygon của HighRes Meshes

Mục tiêu của phương pháp là đạt: $N(\text{Adaptive}) \ll N(\text{High})$

trong khi sai số hình học tại ROI vẫn nằm dưới ngưỡng cho phép: $\text{Error}_{\text{ROI}} \leq \tau_{\text{ROI}}$.

Đặc tả dữ liệu và tổ chức tập dữ liệu:

1. DTU MVS — Bộ dữ liệu chuẩn để đo Chamfer Distance

Vai trò trong đề tài: benchmark chính. Đây là bộ dữ liệu được sử dụng bởi phần lớn các công trình trong literature review (3DGSR, MILO, MeshSplatting, GSurf), nên kết quả của nhóm có thể so sánh trực tiếp với số liệu đã công bố.

Cấu hình thu thập. DTU sử dụng một camera đơn duy nhất gắn trên cánh tay robot công nghiệp sáu trục, di chuyển tới các vị trí đã được hiệu chỉnh chính xác. Camera này đồng thời là một trong các camera của bộ quét structured light, nhờ đó mỗi ảnh RGB đều có một bản quét hình học tương ứng.

Về mặt phân loại, là **single camera di chuyển theo quỹ đạo hiệu chỉnh**. → đảm bảo tính nhất quán tuyệt đối về tham số giữa mọi góc chụp — không có sai lệch do khác biệt cảm biến hay ống kính như trong hệ multi-camera.

Ground truth: Point cloud từ máy quét structured light — độ chính xác cao nhất

Tham số camera: Nội và ngoại đầy đủ, hiệu chỉnh bằng Matlab calibration toolbox

2. OmniObject3D — Bộ dữ liệu quét thật với ground truth và ảnh sẵn có

Vai trò trong đề tài: bộ dữ liệu bổ trợ, dùng để kiểm chứng tính khái quát trên nhiều loại vật thể. GSurf — một trong các công trình nền của đề tài — dùng hai subset OmniObject3D-d (8 vật chi tiết) và OO3D-SL (24 vật dưới ánh sáng mạnh).

Đặc điểm. OmniObject3D gồm **6.000 vật thể quét thật thuộc 190 category đời thường**, chia sẻ nhiều lớp chung với các bộ dữ liệu 2D phổ biến như ImageNet và LVIS. Mỗi vật được thu thập bằng **cả cảm biến 2D lẫn 3D**, cung cấp đồng thời mesh có texture, point cloud lấy mẫu, ảnh đa góc nhìn có pose, và video quay thật.

Điểm khác biệt quan trọng so với các bộ dữ liệu tổng hợp: đây là **vật thể quét thật bằng máy quét chuyên dụng**, không phải mô hình CAD dựng tay, nên hình dạng và bề mặt phản ánh đúng đặc tính vật liệu thực tế.